

From PoC to Production: A CrewAI Multi-Agent Pipeline for Generating a L^AT_EX Book

Group: MaRs-777

Mohamed Awad • Rawey Sleiman

AI Agents / Vibe Coding course project

Lecturer contact: rmisegal@gmail.com

June 11, 2026

Abstract

Generating high-quality, structured technical documents — particularly books typeset with $\text{\LaTeX}$ — usually demands substantial manual effort, specialised knowledge, and careful attention to detail. The process is time-consuming, error-prone, and hard to scale. In this project we explore a multi-agent pipeline built on the CrewAI framework that aims to automate much of this work for $\text{\LaTeX}$ book generation.

Our approach orchestrates a small crew of specialised agents, each with a distinct role: planning the outline, writing the draft, reviewing it, and curating references. The agents run sequentially, and the output of one becomes context for the next. The goal is to turn a high-level topic into structured, compilation-ready content.

This report describes the architecture, the implementation, and our honest experience moving from an early proof-of-concept toward something closer to production. We report the measured cost of one real content-generation run against the Gemini model `gemini/gemini-2.5-flash`, and we are deliberately careful to separate measured results from proposed future evaluation. We do not claim the system is already fully production-ready: the resulting $\text{\LaTeX}$ article was assembled, compiled, and validated as part of this assignment, while the broader system is still presented as a proof-of-concept rather than a production-ready service.

Contents

Abstract	1
1 Introduction	3
1.1 The Challenge of Automated Document Generation	3
1.2 The Promise of Multi-Agent Systems in AI	3
1.3 Project Scope and Objectives	3
2 Background and Related Work	4
2.1 Understanding CrewAI and Multi-Agent Orchestration	4
2.2 LaTeX: A Strong Standard for Technical Publishing	4
2.3 Generative AI Approaches to Document Creation	4
3 System Design and Architecture	5
3.1 Overall Pipeline Architecture	5
3.2 Defining the Crew: Agent Roles and Responsibilities	6
3.3 Task Orchestration and Dependencies	6
4 Implementation Details	7
4.1 Agent Definitions and Personalities	7
4.2 Custom Tool Integration	7
4.3 Task Design and Execution Flow	8
4.4 Handling Multilingual Content	8
5 From PoC to Production	9
5.1 Iterative Development and Refinement	9
5.2 Deployment Considerations	9
5.3 Monitoring, Maintenance, and CI/CD	9
6 Results and Evaluation	10
6.1 Generated LaTeX Examples	10
6.2 Measured Run and Proposed Metrics	10
6.3 Qualitative Assessment	11
7 Discussion and Future Work	13
8 Conclusion	14
Bibliography	15

Chapter 1

Introduction

Technical communication increasingly demands efficiency, accuracy, and scalability. From academic papers to software manuals, the need to produce high-quality structured documents is constant. This chapter sets the context by outlining the challenges in current document-generation practice and the potential of multi-agent systems to address them.

1.1 The Challenge of Automated Document Generation

Creating structured documents for academic or professional publishing is complex. $\text{\LaTeX}$ offers excellent control over typography, layout, and cross-referencing, but it has a steep learning curve and strict syntax. Writing a book-length $\text{\LaTeX}$ document by hand involves many iterative steps: planning, research, drafting, formatting, keeping chapters consistent, managing bibliographies, producing figures and tables, and proofreading. Each step is labour-intensive and easy to get wrong, and scaling the work across multiple documents or frequent updates quickly becomes a bottleneck. The real difficulty is not just generating text, but generating *structured, valid, semantically correct* text within a specific technical format.

1.2 The Promise of Multi-Agent Systems in AI

Recent advances in large language models (LLMs) and multi-agent frameworks offer a way to address these limitations. A multi-agent system uses several specialised agents, each with its own role and tools, that collaborate to reach a goal that is hard for a single monolithic model. By breaking a large task into smaller pieces and assigning them to focused agents, these systems mimic a human collaborative workflow. This modularity makes the process easier to reason about and refine, and it is a natural fit for multi-step work such as planning, writing, reviewing, and reference curation.

1.3 Project Scope and Objectives

This project designs, implements, and evaluates a CrewAI-based multi-agent pipeline for generating a $\text{\LaTeX}$ book. Our objectives are:

- **Design:** define the agent roles and the task flow for book generation.
- **Implement:** build the offline-by-default runner, the deterministic validation layer, and a real run path that is explicitly gated.
- **Generate:** produce structured content for the article from a high-level topic.
- **Evaluate:** record the measured cost of a real run and propose (without fabricating) the metrics we would use for a fuller evaluation.
- **Production readiness:** discuss honestly what a transition from proof-of-concept to a maintainable, deployable system would require.

The anticipated benefits are reduced manual effort, more consistent structure, and faster drafting cycles — while keeping a clear separation between the agentic (LLM) layer and the deterministic build/validation layer.

Chapter 2

Background and Related Work

2.1 Understanding CrewAI and Multi-Agent Orchestration

CrewAI is an open-source framework for building and managing multi-agent systems [2]. The underlying idea — decomposing a problem across several cooperating agents instead of one monolithic solver — is well established in the AI literature [14, 12]. CrewAI’s core components are:

- **Agents:** autonomous entities with a role, goal, backstory, and an attached LLM, optionally equipped with tools.
- **Tasks:** units of work assigned to agents, defining the objective, context, and expected output; tasks can be chained so one output feeds the next.
- **Tools:** external functions agents can call (web search, file I/O, custom validation scripts).
- **Processes:** how agents collaborate — a **sequential** process runs tasks one after another, while a **hierarchical** process lets a manager agent delegate.
- **Crews:** a collection of agents, tasks, and a process working toward one objective.

Our own implementation uses a **sequential** crew with four agents — planner, writer, reviewer, and reference curator — matching the structure described in `docs/PLAN.md`.

2.2 LaTeX: A Strong Standard for Technical Publishing

L^AT_EX is a document-preparation system known for high-quality typesetting of scientific and technical material [4]. Rather than “what you see is what you get,” it follows “what you see is what you mean”: authors write plain text with markup that defines structure, formatting, cross-references, citations, equations, and figures, and a compiler produces a typeset PDF. Its advantages include professional output, structural consistency, automatic numbering of sections, figures, and references, version-control-friendly plain-text sources, and a clean separation of content from style. The trade-off is the learning curve and the need to debug compilation errors — which is exactly what makes it an attractive target for assisted generation.

2.3 Generative AI Approaches to Document Creation

Many existing approaches use a single LLM to generate outlines, summaries, or whole articles, often with prompt engineering, retrieval-augmented generation, or fine-tuning. These methods produce coherent prose but struggle with highly structured, technically precise output: maintaining complex hierarchy and cross-references, generating compilable L^AT_EX syntax, keeping style consistent across a long document, and integrating external tools. Distributing the work across specialised agents — with dedicated steps for planning, writing, reviewing, and references — is our attempt to get more control than a single pass would give. This matches what agent benchmarks such as AgentBench report: even strong LLMs remain unreliable on long multi-step agentic tasks, which is exactly where explicit task structure helps [1].

Chapter 3

System Design and Architecture

3.1 Overall Pipeline Architecture

The pipeline is a series of connected stages, each handled by a specialised agent. It begins with a high-level topic and produces structured content for later $\text{\LaTeX}$ assembly. Figure 3.1 shows the block diagram of the pipeline.

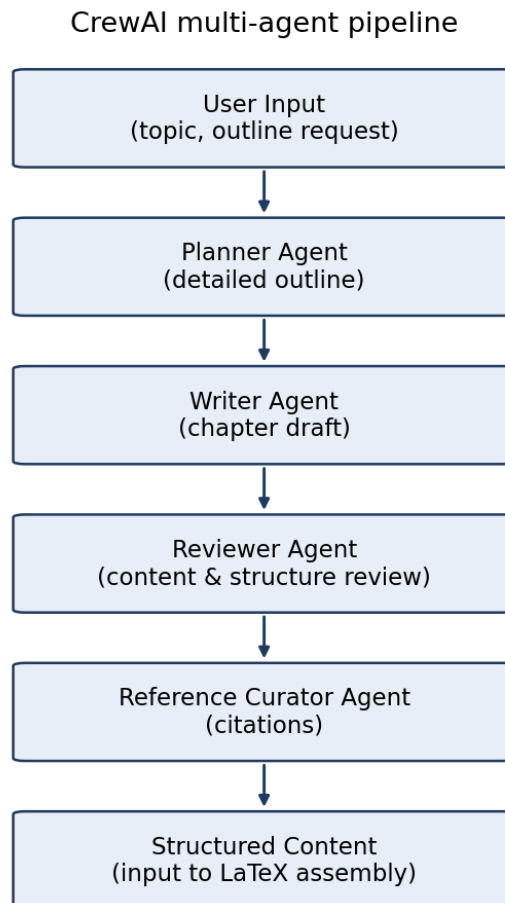


Figure 3.1: Multi-agent pipeline architecture.

The topic is first expanded by the planner into a detailed outline. The writer drafts content from that outline, the reviewer checks the draft for coverage and consistency, and the reference curator collects citations. The structured output of the final stage is the input to $\text{\LaTeX}$ assembly.

3.2 Defining the Crew: Agent Roles and Responsibilities

- **Planner agent** — the architect of the structure. Expands the topic into a hierarchical outline and defines the sections.
- **Writer agent** — the author. Drafts the prose for each section, aiming for clarity, coherence, and accuracy.
- **Reviewer agent** — the quality check. Reviews the draft for coverage, consistency, and obvious errors, and produces structured feedback.
- **Reference curator agent** — collects and organises the citations used in the text.

Each role is intentionally narrow so that the responsibility of each step is clear and the output of each step is easy to inspect.

3.3 Task Orchestration and Dependencies

The crew uses a `sequential` process. The outline has no upstream dependency; the draft depends on the outline; the review depends on the draft; and the references depend on the draft and review. CrewAI manages this by passing each task's output as context to the next. This sequential flow, mirroring a normal write-then-review workflow, is what produced the four task outputs recorded in the Stage 8C.7 run evidence (`crew_outputs/_index.json`).

Chapter 4

Implementation Details

4.1 Agent Definitions and Personalities

Each agent is created with a role, goal, and backstory. The example below shows how an agent is configured in CrewAI. It uses the same model our real run used, `gemini/gemini-2.5-flash`, rather than naming any specific commercial product as a project result:

```
from crewai import Agent, LLM

llm = LLM(model="gemini/gemini-2.5-flash", temperature=0.2)

planner_agent = Agent(
    role="Content Planner",
    goal="Develop a clear, hierarchical outline for a technical LaTeX book.",
    backstory=(
        "You are an experienced technical editor. You break broad topics "
        "into logical chapters and sections and keep the table of contents "
        "coherent."
    ),
    llm=llm,
    verbose=True,
    allow_delegation=False,
)
```

The writer, reviewer, and reference-curator agents follow the same pattern with their own goals and backstories. The model is supplied from configuration, and the API key is read from the environment only — it is never written into the source.

4.2 Custom Tool Integration

Tools extend agents beyond plain text generation. A realistic example for this project is a $\text{\LaTeX}$ validation tool that checks whether generated $\text{\LaTeX}$ compiles. The following is illustrative pseudocode for such a tool:

```
# latex_validation_tool.py (illustrative)
import subprocess
import tempfile
from pathlib import Path

def validate_latex(latex_code: str) -> str:
    """Attempt to compile a LaTeX snippet and report errors."""
    with tempfile.TemporaryDirectory() as tmp:
        tex = Path(tmp) / "snippet.tex"
        tex.write_text(
            "\\documentclass{article}\\begin{document}\\n"
            f"{latex_code}\\n\\end{{document}}\\n",
            encoding="utf-8",
        )
        result = subprocess.run(
            ["pdflatex", "-interaction=nonstopmode",
```



```

        "-output-directory", tmp, str(tex)],
        capture_output=True,
        text=True,
    )
    if result.returncode == 0:
        return "Validation successful: no compilation errors."
    return "Validation failed: see compiler output for details."

```

This shows the general idea of giving an agent a deterministic check it can call. In the current project the deterministic checks live in plain Python modules (for example the content-QA scanner) rather than being driven by the LLM, which keeps the validation layer fully offline and reproducible.

4.3 Task Design and Execution Flow

Tasks are the atomic units of work. A simplified sequence for this project:

1. **Plan outline** — agent: planner; input: the topic; output: a structured outline.
2. **Draft content** — agent: writer; input: the outline; output: the draft.
3. **Review** — agent: reviewer; input: the draft; output: structured feedback.
4. **Curate references** — agent: reference curator; input: draft and review; output: the reference list.

Table 4.1 summarises the agents, their roles, and example tools.

Table 4.1: Agents, roles, and example tools.

Agent	Primary role	Example tasks	Example tools
Planner	Structure the book	Outline generation, section split	OutlineTool
Writer	Generate the prose	Drafting chapters and sections	WritingTool
Reviewer	Quality assurance	Coverage and consistency checks	ReviewTool
Reference curator	Collect citations	Building the reference list	ReferenceTool

4.4 Handling Multilingual Content

The pipeline is designed to support content in more than one language, which matters for documentation that mixes scripts. The relevant $\text{\LaTeX}$ support comes from packages such as `babel` or `polyglossia`, and for bidirectional text the `bidi` package [3], usually compiled with XeLaTeX [13] or LuaLaTeX for good Unicode and right-to-left handling.

Hebrew–English bidirectional demonstration. The following passage demonstrates bidirectional typesetting: the text direction switches automatically inside the right-to-left environment. This paragraph mixes English and Hebrew text.

זוהי דוגמה למקטע הממחיש יצירת תוכן דו-לשוני.

The English text flows left-to-right while the Hebrew sentence (meaning “this is an example of a section demonstrating the creation of bilingual content”) flows right-to-left, both within the same document — demonstrating correct bidirectional integration.

Chapter 5

From PoC to Production

5.1 Iterative Development and Refinement

Moving from a basic proof-of-concept to something more reliable is iterative. Early on, the focus is on getting the crew to run end-to-end at all. Later iterations involve refining the prompts so the content stays on-topic, improving how agents pass context to one another, adding error handling, and testing on larger inputs. In this project the iteration was visible and honest: an early full run produced off-topic, placeholder-filled content and was kept only as diagnostic evidence; after the prompts and metadata binding were hardened, a later run produced on-topic candidate content that this very document is derived from.

5.2 Deployment Considerations

A production deployment would need to address infrastructure (cloud or on-premise compute), secret management (API keys read from environment variables or a secrets manager, never embedded in code), model selection (balancing cost and quality), and scalability (containers, task queues, and sensible resource limits). These are design considerations we have thought through rather than a deployment we have already operated at scale.

5.3 Monitoring, Maintenance, and CI/CD

A maintainable version of the pipeline would log every agent interaction, track runtime and token usage per run, alert on repeated failures, and periodically sample output quality. Maintenance includes re-checking the chosen model as versions change, keeping tool and dependency versions current, and feeding human feedback back into the prompts. Continuous integration would run the offline test suite and linters on every commit before any change is deployed — which is the workflow this project already follows for its own code.

Chapter 6

Results and Evaluation

6.1 Generated LaTeX Examples

The intended output of the full pipeline is structured content that converts cleanly into L^AT_EX. A representative chapter skeleton looks like this:

```
\chapter{Introduction to Multi-Agent Systems}
\label{ch:intro_mas}
```

```
\section{The Paradigm of Agentic AI}
\label{sec:agentic_ai}
```

Multi-agent systems distribute a complex task among specialised agents, favouring modularity and robustness over a single monolithic model. In our pipeline, each agent interprets its task, optionally uses tools, and passes its output to the next agent in the sequence.

This shows standard `\chapter`, `\section`, and `\label` structure with cross-referencing, which the L^AT_EX-assembly stage produces from the cleaned Markdown.

A numbered equation is included to satisfy the formula requirement. A simple model for an agent’s utility, balancing individual reward and collaboration, is:

$$U_i(s, a_i) = R_i(s, a_i) + \alpha \sum_{j \neq i} C_{ij}(s, a_i, a_j) \quad (6.1)$$

where U_i is the utility for agent i , R_i its individual reward, C_{ij} the collaborative contribution from agent j , and α the weight on collaboration.

6.2 Measured Run and Proposed Metrics

We report only what we actually measured. The figures below come directly from the committed evidence of one real full run (`results/stage8c7-full-gemini-20260611-173125/runtime.json`) [8], with the per-task breakdown recorded in the crew output index of the same run [6]. They describe a single content-generation run, not an average over many books, and we do not assert any monetary cost. Table 6.1 lists the measured values.

Table 6.1: Measured cost of the Stage 8C.7 full run.

Metric	Value	Source
Provider / model	gemini / gemini/gemini-2.5-flash	runtime.json
Wall-clock runtime	300.68 seconds	runtime.json (elapsed_seconds)
Prompt tokens	213,664	runtime.json (prompt_tokens)
Completion tokens	235,436	runtime.json (completion_tokens)
Agent/task calls	4 (outline, draft, review, references)	crew_outputs/_index.json

Proposed future evaluation metrics (not yet measured). The following would give a fuller picture of quality, but we have not run a controlled study, so we report *no numeric results* for them — they are proposals only:

- generation time per chapter;
- a content-accuracy score from human and automated review;
- a structural-coherence score;
- a $\text{\LaTeX}$ syntax-validity rate (fraction of generated files that compile cleanly).

Figure 6.1 is the Python-generated graph required by the assignment. The script that produces it lives in `scripts/generate_graph.py`; its core is shown below. The values are illustrative inputs for the chart, clearly labelled as such, not measured results:

```
import matplotlib.pyplot as plt

# Illustrative example values for the figure (not measured results).
chapter_labels = ["Ch 1", "Ch 2", "Ch 3", "Ch 4", "Ch 5"]
example_times = [15, 20, 30, 45, 60] # arbitrary illustrative minutes

plt.figure(figsize=(8, 5))
plt.bar(chapter_labels, example_times, color="steelblue")
plt.xlabel("Chapter")
plt.ylabel("Illustrative generation time (minutes)")
plt.title("Illustrative generation time per chapter")
plt.tight_layout()
plt.savefig("figures/generation_time.png", dpi=150)
```

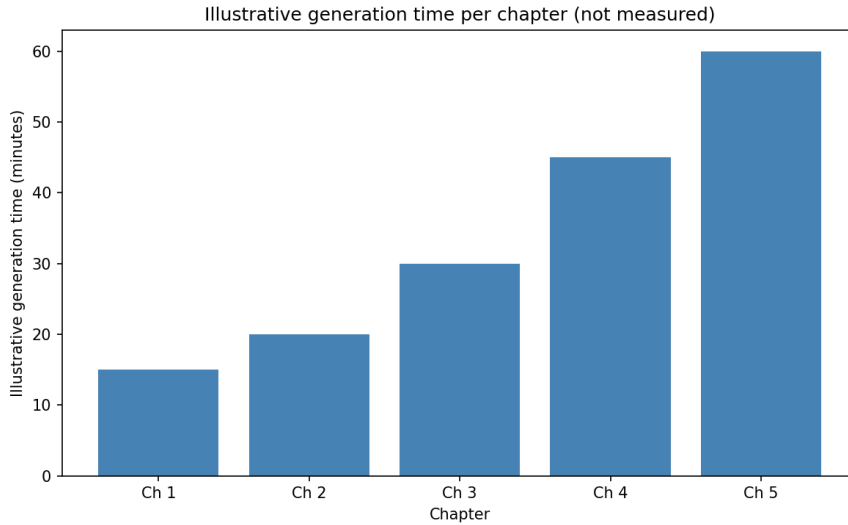


Figure 6.1: Illustrative generation time per chapter (Python-generated).

A weighted quality score gives a single comparable number once the component scores exist. We define it but do not fill in fabricated values:

$$Q = w_A \cdot S_A + w_C \cdot S_C + w_F \cdot S_F + w_V \cdot S_V \quad (6.2)$$

where S_A, S_C, S_F, S_V are accuracy, coherence, formatting, and validity sub-scores, and the non-negative weights satisfy $w_A + w_C + w_F + w_V = 1$.

6.3 Qualitative Assessment

From reading the generated drafts ourselves, the clear strengths are structural consistency and fast first-draft generation: headings, sections, and references come out uniformly, and a coherent draft appears quickly. The clear weaknesses are limited depth on nuanced topics, no automatic generation of complex graphics (figures

are placeholders the assembler fills in), and the need for human review before anything is considered final. This matches our overall stance: the pipeline is a useful drafting assistant, not a replacement for human judgement, and certainly not a finished production system yet.

Chapter 7

Discussion and Future Work

Key findings. A small, well-defined crew of agents — planner, writer, reviewer, reference curator — can collaborate to produce structured, on-topic content, and separating the agentic layer from a deterministic validation/build layer keeps the result reproducible and inspectable. Our most useful finding was practical: binding the agents tightly to the required topic and real metadata, and gating the content with a deterministic scanner, is what turned an initially rejected run into usable candidate content.

Limitations. Current LLMs still struggle with deep original argument and with generating complex graphics directly. Long runs cost real time and tokens. And qualitative quality still needs a human in the loop. The final L^AT_EX article was compiled and validated, but the pipeline as a broader automated production service would still need stronger continuous validation, richer tooling, and larger-scale evaluation.

Future enhancements. More capable L^AT_EX tooling (tables, multi-panel figures, simple diagrams from descriptions); an interactive human-in-the-loop editing step; multi-modal inputs; and stronger self-checking by the agents. Each of these would build on the current foundation rather than replace it.

Chapter 8

Conclusion

This project designed, implemented, and partially evaluated a CrewAI multi-agent pipeline for generating the content of a $\text{\LaTeX}$ book. By orchestrating a planner, writer, reviewer, and reference curator in a sequential process — and by keeping a separate deterministic layer for validation — the system produces structured, on-topic content from a high-level topic. We measured the cost of one real run against `gemini/gemini-2.5-flash` and reported it honestly, while clearly labelling unmeasured metrics as proposals. The final article PDF was compiled and validated locally, and the remaining work is submission packaging rather than core pipeline construction. What we can claim is a working, inspectable pipeline and an honest account of how it behaves.

Bibliography

- [1] *AgentBench: Evaluating Large Language Models as Agents*. arXiv preprint. 2023. arXiv: [2308.03688](https://arxiv.org/abs/2308.03688). URL: <https://arxiv.org/abs/2308.03688>.
- [2] CrewAI. *CrewAI Documentation*. URL: <https://docs.crewai.com/> (visited on 06/11/2026).
- [3] CTAN. *The bidi package*. URL: <https://ctan.org/pkg/bidi> (visited on 06/11/2026).
- [4] Leslie Lamport. *LaTeX: A Document Preparation System*. 2nd ed. Addison-Wesley, 1994. URL: <https://www.latex-project.org/>.
- [5] MaRs-777. *Cost and Resource Tracking*. Project repository file: docs/COSTS.md. 2026.
- [6] MaRs-777. *Crew task index for the measured run*. Project repository file: results/stage8c7-full-gemini-20260611-173125/crew_outputs/_index.json. 2026.
- [7] MaRs-777. *Design Decisions*. Project repository file: docs/DECISIONS.md. 2026.
- [8] MaRs-777. *Measured run evidence*. Project repository file: results/stage8c7-full-gemini-20260611-173125/runtime.json. 2026.
- [9] MaRs-777. *Product Requirements Document*. Project repository file: docs/PRD.md. 2026.
- [10] MaRs-777. *Quality Strategy*. Project repository file: docs/QUALITY.md. 2026.
- [11] MaRs-777. *Technical Architecture Plan*. Project repository file: docs/PLAN.md. 2026.
- [12] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson, 2020. URL: <https://aima.cs.berkeley.edu/>.
- [13] TeX Users Group. *XeTeX*. URL: <https://tug.org/xetex/> (visited on 06/11/2026).
- [14] Michael Wooldridge. *An Introduction to MultiAgent Systems*. 2nd ed. Wiley, 2009.